

Triton vs. CUDA: A Workload-Driven Performance Analysis on Modern GPUs

*Focus on Structured vs. Irregular (FSM) Workloads

Alessandro Potenza
NECST Laboratory, DEIB
Politecnico di Milano
Milano, Italy
alessandro1.potenza@mail.polimi.it

Davide Conficconi
NECST Laboratory, DEIB
Politecnico di Milano
Milano, Italy
davide.conficconi@mail.polimi.it

Filippo Del Nero
NECST Laboratory, DEIB
Politecnico di Milano
Milano, Italy
filippo.delnero@mail.polimi.it

Abstract—GPUs have become the engine of modern high-performance computing. While NVIDIA CUDA remains the industry standard, its complexity creates a high barrier to entry. OpenAI Triton, a Python-based DSL, aims to democratize GPU programming through a block-based model. While Triton excels at dense deep learning primitives, its capability to handle irregular, latency-bound workloads remains an open question.

This paper presents a workload-driven performance analysis comparing Triton and CUDA across two domains: structured, data-parallel computations (Matrix Multiplication) and irregular, control-flow intensive tasks (Finite State Machine simulation). Our results show that for structured workloads, Triton achieves performance parity with CUDA. For irregular workloads involving pointer-chasing and divergent control flow, CUDA kernels are 10–30× faster than the best Triton implementations. We conclude that while Triton increases productivity for dense linear algebra, low-level CUDA remains essential for high-performance irregular algorithms.

Index Terms—GPGPU, CUDA, Triton, Finite State Machines, Irregular Workloads, Performance Analysis

I. INTRODUCTION

GPUs have become an indispensable driving force behind a wide range of applications. NVIDIA’s CUDA [1] has dominated this environment for over a decade, providing a programming model that exposes low-level hardware details. This results in maximum theoretical performance but presents a significant barrier to entry due to its complexity.

In response to this productivity challenge, higher-level abstractions have emerged. Among the most popular is Triton [2], a Python-based DSL developed by OpenAI. Triton seeks to democratize GPU programming by allowing developers to write computations on data blocks, letting its JIT compiler handle memory coalescing and shared memory management. While Triton has proven successful for dense neural network layers, this poses a challenging question: *To what extent can a high-level block-based abstraction rival the performance of low-level CUDA across different computational problems?*

This paper addresses this question through a workload-driven comparative study. We benchmark two radically different types of workloads:

- **Structured Workloads:** Problems like Matrix Multiplication, characterized by uniform data access and deterministic control flow.
- **Irregular Workloads:** Problems like Finite State Machine (FSM) simulation, characterized by sparse memory accesses (pointer-chasing) and divergent control flow.

The main contributions of this paper are:

- 1) A performance comparison of Triton and CUDA kernels on both structured (MatMul) and irregular (FSM) computational domains.
- 2) An implementation of specialized FSM kernels in Triton (Bitmap, CSR, DFS) compared against CUDA approaches.
- 3) An analysis of the performance gap observed in irregular workloads, relating them to the programming model design.

Our principal thesis is that the optimal programming model is workload-dependent. While Triton achieves performance parity for structured computations, the low-level control offered by CUDA is essential for irregular workloads. We show that for FSM simulation, CUDA kernels outperform Triton by 10–30× due to the latter’s inability to efficiently handle fine-grained latency-bound operations.

II. BACKGROUND

To understand the performance analysis presented in this work, it is essential to first grasp the fundamental differences between the programming models under investigation and the characteristics of the workloads used for evaluation.

A. GPU Programming Models

1) *CUDA C++ (The Baseline):* NVIDIA’s CUDA [1] adopts a Single Instruction Multiple Thread (SIMT) execution model. It grants developers direct and explicit control over the GPU architecture. Programmers manage the thread hierarchy (grid, blocks, and warps), handle memory transfers between host and device, and manually optimize memory access patterns to maximize bandwidth (e.g., ensuring coalesced global

memory reads). This fine-grained control allows for sophisticated optimizations, such as using *Warp Shuffle* primitives for register-level communication or managing Shared Memory to reduce global traffic. However, this comes at the cost of high code verbosity and complexity.

2) *OpenAI Triton (The Abstraction)*: Triton [2] introduces a block-based programming model. Instead of writing code for a single thread (as in CUDA), the developer writes operations on blocks of data (tensors). The Triton compiler automatically handles the complex tasks of multi-threading, shared memory allocation, and synchronization barriers. While this abstraction simplifies the development of dense matrix operations, it hides the concept of an individual thread ID. This makes it difficult to express algorithms that require per-thread logic or random access patterns, as operations must be defined over entire blocks.

B. Workload Characterization

The success of a GPU programming model relies significantly on the nature of the target workload. We classify our benchmarks into two broad categories:

1) *Structured Workloads*: These applications represent the classical use case for GPUs, such as Dense Neural Network layers and Matrix Multiplication. They are characterized by:

- **Predictable Memory Access**: Threads read memory in contiguous sequences, enabling fully coalesced loads.
- **High Arithmetic Intensity**: A high ratio of compute operations to memory references.
- **Uniform Control Flow**: All threads in a warp typically follow the same execution path.

These characteristics map perfectly to Triton’s block-level semantics.

2) *Irregular Workloads*: This class of problems exposes the limits of parallel architectures. We utilize Finite State Machine (FSM) simulation as our representative irregular workload. FSM processing is characterized by:

- **Pointer Chasing**: To evaluate the next state, the algorithm must read a value from a memory location determined by the current state ($S_{t+1} = \delta(S_t, Input)$). This results in scattered, non-coalesced memory accesses.
- **Sequential Dependency**: The next state depends entirely on the previous one, creating a strict latency-bound chain that is hard to pipeline.
- **Divergent Control Flow**: Different input strings may lead to different states, causing threads in a warp to diverge and serialize execution.

III. METHODOLOGY

This section details the experimental setup, the system configuration, and the specific algorithms implemented to compare the programming models.

A. System Configuration

All benchmarks were executed on a consistent hardware platform to ensure fair comparisons.

- **GPU**: NVIDIA GeForce RTX 4070 (Ada Lovelace, 12GB GDDR6X).
- **Software Stack**: CUDA Toolkit 12.3, PyTorch 2.1, Triton 2.1.
- **Environment**: A strictly controlled Docker container to ensure reproducibility of the Python/C++ build environment.

B. Algorithm Implementation Details

Our core investigation focuses on the contrast between high-level and low-level implementations of FSM simulation.

1) *Triton Implementation (High-Level)*: The Triton kernel adopts a “dense” approach to fit the block-based programming model. It tracks active states using a boolean tensor or sparse list, but relies on masked loads to handle the irregularity of the transition table. As shown in Algorithm 1, the absence of per-thread branching forces the compiler to emit predicated instructions that execute both the memory load and the validity check, often reading unnecessary data (over-fetching).

Algorithm 1 Triton CSR Kernel Logic (Simplified)

```

1: Input: state_id, input_char
2: row_start  $\leftarrow$  tl.load(row_ptr + state_id)
3: row_end  $\leftarrow$  tl.load(row_ptr + state_id + 1)
4: curr  $\leftarrow$  row_start
5: while curr < row_end do
6:   // Masked Load required for irregular access
7:   sym  $\leftarrow$  tl.load(data + curr, mask = curr < row_end)
8:   target  $\leftarrow$  tl.load(indices + curr, mask = curr < row_end)
9:   if sym == input_char then
10:    tl.atomic_store(next_state, target)
11:   end if
12:   curr  $\leftarrow$  curr + 1
13: end while

```

2) *CUDA Implementation (Low-Level)*: The CUDA kernel implements the *ngAP* (N-Grams Automata Processor) strategy [3]. It utilizes a global worklist to balance load and specialized hardware intrinsics. As shown in Algorithm 2, we explicitly manage the cache hierarchy using the ‘__ldg’ intrinsic (read-only cache) and reduce global memory contention by aggregating atomic updates at the warp level using ‘__ballot_sync’.

C. Benchmark Metrics

We evaluate performance using the following metrics:

- **Kernel Execution Time (ms)**: The raw time spent by the GPU to execute the compute kernel.
- **System Overhead**: The time spent in driver latency, JIT compilation, and data movement, calculated as $T_{Total} - T_{Kernel}$.
- **Speedup**: Defined as T_{Triton}/T_{CUDA} .

Algorithm 2 CUDA ngAP Kernel Logic (Optimized)

```

1: Input: worklist (Shared Memory / Global)
2: state_id  $\leftarrow$  pop_work_item(worklist)
3: // Read-Only Cache Optimization
4: row_start  $\leftarrow$  __ldg(row_ptr + state_id)
5:
6: for i  $\leftarrow$  row_start to row_end do
7:   sym  $\leftarrow$  __ldg(data + i)
8:   if sym == input_char then
9:     next  $\leftarrow$  __ldg(indices + i)
10:    // Warp Aggregation to reduce global atomics
11:    vote  $\leftarrow$  __ballot_sync(FULL_MASK, true)
12:    if lane_id == __ffs(vote) - 1 then
13:      atomicAdd(global_queue, popc(vote))
14:    end if
15:  end if
16: end for

```

IV. RESULTS AND ANALYSIS

This section presents our experimental results across four dimensions: structured workload parity, the irregular workload performance gap, CUDA technique comparison, and the productivity-performance trade-off.

A. Structured Workloads: Parity

For structured, data-parallel workloads, Triton delivers on its promise. Fig. 1 shows results for Matrix Multiplication and Multi-Layer Perceptron (MLP) forward passes.

Matrix Multiplication. Triton achieves near-parity with cuBLAS across matrix sizes. At 1024^3 , Triton is $1.08\times$ slower (0.156 ms vs 0.145 ms). At 2048^3 , Triton matches cuBLAS ($0.99\times$, 0.959 ms vs 0.967 ms). At 4096^3 , the gap widens to $1.27\times$ (8.06 ms vs 6.35 ms), likely due to cache optimization in cuBLAS.

MLP Forward Pass. For MLP workloads, Triton achieves parity or better ($0.75\times$ – $1.20\times$). The best Triton implementations leverage kernel fusion, avoiding intermediate memory writes between layers. For example, at configuration $512\times 2048\times 4096\times 1024$, fused Triton achieves 0.58 ms compared to 0.77 ms for cuBLAS-based CUDA.

These results confirm that Triton’s block-based abstraction maps well to hardware tensor cores for dense algebra with negligible abstraction cost.

B. Irregular Workloads: The Gap

The scenario changes for Finite State Machines (FSM). Fig. 2 shows the performance gap on a logarithmic scale.

CUDA implementations outperform Triton by 10 – $30\times$ (median: $10\times$). This gap is structural:

- **Latency-Bound:** FSM processing requires reading single bytes at random locations. Triton’s `tl.load` fetches blocks, causing bus under-utilization.
- **Control Flow:** Triton handles conditionals via masking. CUDA skips unnecessary work with thread-local logic.

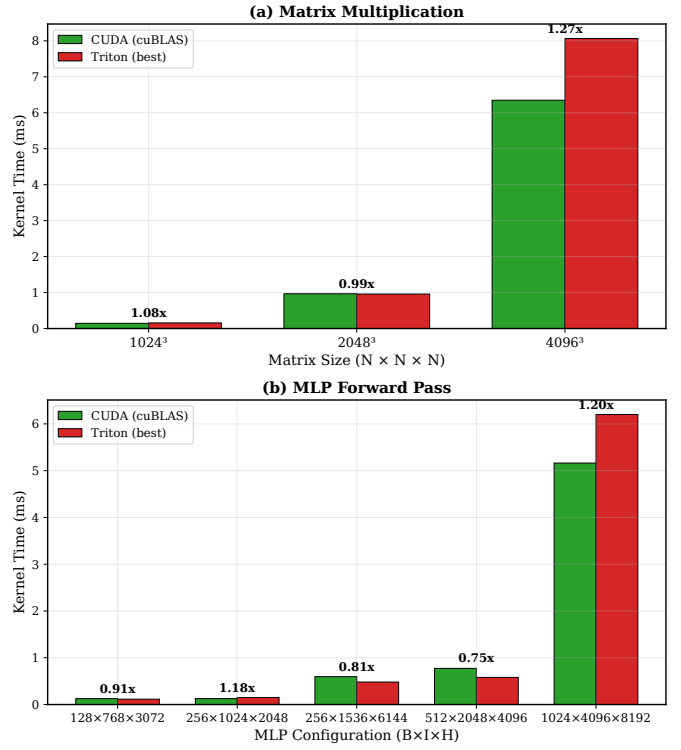


Fig. 1. Structured Workloads. (a) Matrix Multiplication: Triton achieves $0.99\times$ – $1.27\times$ vs cuBLAS. (b) MLP Forward Pass: Triton achieves $0.75\times$ – $1.20\times$, with kernel fusion providing speedups on larger models.

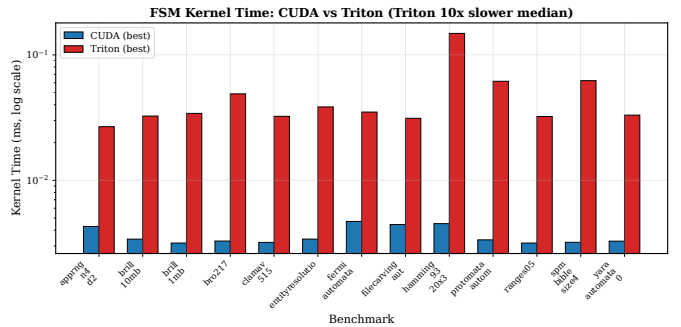


Fig. 2. The Irregularity Gap. CUDA kernels are 10 – $30\times$ faster than Triton for FSM processing.

C. Technique Comparison: CUDA vs Triton

Fig. 3 compares FSM techniques from both CUDA and Triton across all benchmarks. CUDA techniques include Basic, CSR, ngAP, and BitGen. Triton techniques include DFS (best), CSR, and Bitmap.

Key findings:

- **CUDA Basic/CSR:** Fastest overall (3 – $6\ \mu\text{s}$ median). Simple serial traversal is effective when memory layout is optimized.
- **CUDA BitGen:** Median $0.14\ \text{ms}$, slower than Basic but transforms memory-bound to compute-bound workloads.
- **Triton DFS:** Best Triton technique ($34\ \mu\text{s}$ median), but still $10\times$ slower than CUDA Basic.

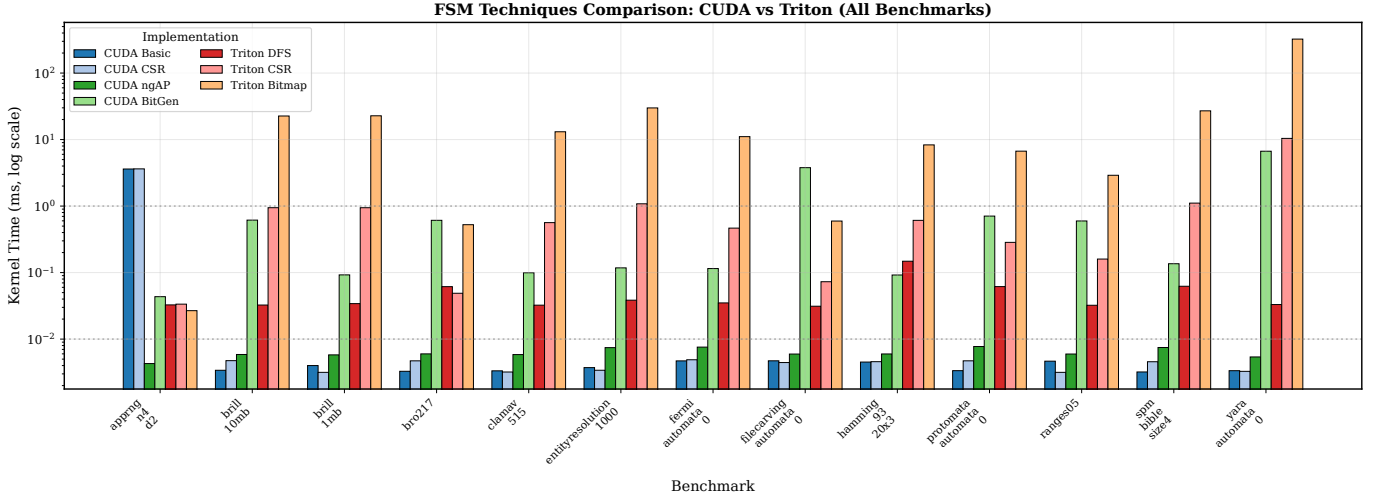


Fig. 3. FSM Techniques Comparison: CUDA vs Triton. Blue/green bars show CUDA techniques (3–140 μ s). Red/orange bars show Triton techniques (34 μ s–15 ms). The gap is consistent across all benchmarks.

- **Triton Bitmap:** Slowest approach (11–15 ms), 1000 $\times$ slower than CUDA Basic.

The gap between CUDA and Triton is consistent across all benchmarks and techniques, confirming the structural nature of the performance difference.

D. Speedup Analysis

Fig. 4 shows the CUDA speedup over Triton for each FSM benchmark, sorted by magnitude. The speedup ranges from $6.2\times$ to $32.8\times$, with a median of $10.2\times$.

The benchmarks with highest speedup (brill, protomata, yara) involve complex automata with many states and transitions. The benchmarks with lower speedup (apprng, filecarving) have simpler structures where Triton’s overhead is less pronounced.

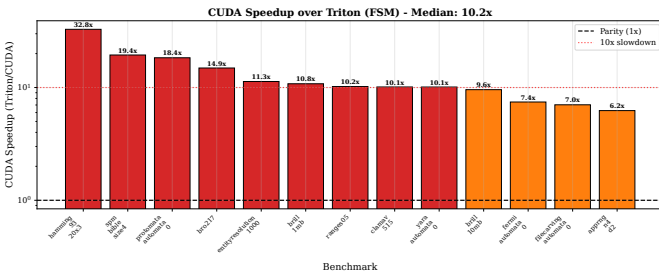


Fig. 4. CUDA Speedup over Triton (FSM). Benchmarks sorted by speedup magnitude. Red bars indicate $> 10\times$ slowdown, orange 5–10 $\times$, green $< 5\times$.

E. Productivity vs Performance Trade-off

Fig. 5 plots development effort (Lines of Code) against performance.

- **Triton:** 45–60 LoC, lowest performance for FSM.
- **CUDA Basic/CSR:** 120–180 LoC, best performance for single-stream processing.

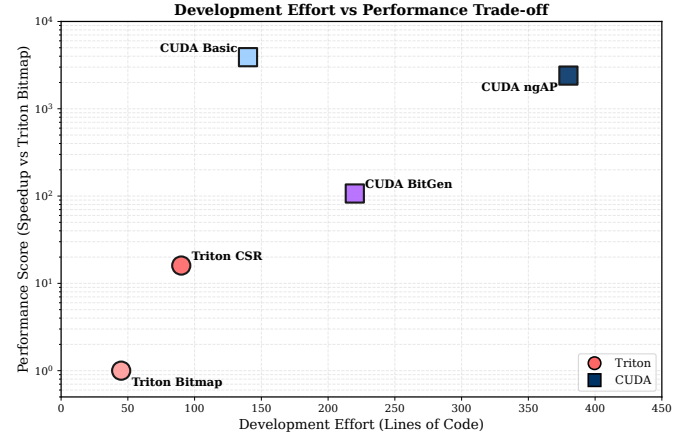


Fig. 5. Productivity vs Performance. Triton allows concise code but hits a performance ceiling. CUDA techniques span a range of complexity and performance.

- **CUDA BitGen:** ≈ 250 LoC, competitive for multi-pattern workloads.
- **CUDA ngAP:** ≈ 380 LoC, complex implementation without proportional gains on our benchmark suite.

This demonstrates that for irregular workloads, the abstraction that makes Triton productive prevents expressing necessary low-level optimizations.

F. System Overhead Analysis

Fig. 6 shows the breakdown between kernel execution time and system overhead (PCIe transfer, driver latency) for FSM workloads.

For CUDA Basic/CSR, kernel execution represents less than 0.01% of total time. The remaining 99.99% is system overhead. This has important implications:

- **Batching:** For production workloads, batching multiple FSM operations per kernel launch amortizes overhead.

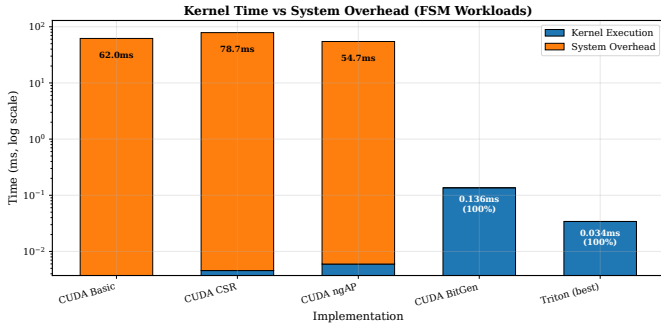


Fig. 6. Kernel Time vs System Overhead. CUDA kernels are so fast (3–6 μ s) that system overhead dominates total execution time.

- **Streaming:** Overlapping data transfer with computation can hide PCIe latency.
- **Triton advantage:** Triton’s higher kernel time means overhead is proportionally smaller, making raw kernel comparisons misleading for end-to-end latency.

V. RELATED WORK

Our research bridges two bodies of literature: high-level GPU programming models and algorithmic optimization of automata processing.

A. High-Level GPU Abstractions

The complexity of CUDA has driven research into Domain-Specific Languages (DSLs). **Triton** [2] simplifies writing high-performance neural network kernels through a tile-based semantics where the compiler manages memory coalescing and shared memory allocation. While Tillet et al. demonstrated Triton’s effectiveness for matrix multiplications, their analysis was confined to dense linear algebra. Our work extends this evaluation to irregular, latency-bound algorithms.

B. Parallel FSM Processing on GPUs

Mapping Finite State Machines to GPUs is difficult due to sequential data dependencies.

1) *Speculative Approaches:* Mytkowicz et al. [4] proposed enumeration-based parallelism, speculatively executing the FSM for all possible starting states. While effective on CPUs with SIMD, this approach has high overhead for single-stream processing.

2) *Memory-Centric Approaches:* Yaneva et al. [5] analyzed memory layouts for FSM traversal, contrasting CSR against dense Transition Matrices. Our Triton implementation builds on these findings with both Bitmap and CSR variants.

3) *Worklist-Based Approaches:* Ge et al. proposed **ngAP** [3], a non-blocking queue-based architecture using warp-level primitives for load balancing. We include ngAP-inspired implementations as a baseline, showing that the required hardware primitives (`warp_shuffle`, fine-grained shared memory) are inexpressible in Triton.

4) *Bit-Parallel Approaches:* Recent work has explored bit-parallel methods that transform regex into bitstream programs. **HybridSA** [6] (OOPSLA 2024) demonstrated CPU-GPU parallel engines using bit parallelism for NFA simulation, reducing irregular memory accesses. **BitGen** [7] (MICRO 2025) introduced interleaved bitstream execution, keeping intermediate data in registers rather than global memory. BitGen reports $19.5\times$ speedup over ngAP by transforming the workload from memory-bound to compute-bound. We evaluate BitGen in our benchmark suite to understand how bit-parallel approaches compare to traditional graph-traversal methods across different automata structures.

VI. CONCLUSION

In this paper, we presented a workload-driven performance analysis comparing OpenAI Triton and NVIDIA CUDA. We sought to determine if a modern block-based DSL could replace low-level code across diverse computational domains.

Our study leads to a nuanced conclusion: the optimal programming model depends on workload regularity.

- **Structured Workloads:** Triton achieves performance parity with cuBLAS and offers significant productivity advantages. For dense algebra, the abstraction cost is effectively zero.
- **Irregular Workloads:** The abstraction breaks down. Block-based semantics introduce significant overhead for latency-bound, pointer-chasing algorithms. CUDA kernels outperform Triton by $10\text{--}30\times$.

We identify this as *Abstraction Regret*: the features that make Triton productive (block abstraction) prevent the expression of micro-optimizations required for irregular domains. Future work should explore extending Triton’s IR to support scalar intrinsics and worklist semantics, potentially bridging this gap.

ACKNOWLEDGMENT

This work was supported by the NECST Laboratory at Politecnico di Milano. We thank the reviewers for their insightful feedback on the architectural analysis of the abstraction gap.

REFERENCES

- [1] M. Garland, S. Le Grand, J. Nickolls, J. Anderson, J. Hardwick, S. Morton, E. Phillips, Y. Zhang, and V. Volkov, “Parallel computing experiences with cuda,” *IEEE Micro*, vol. 28, no. 4, pp. 13–27, 2008.
- [2] P. Tillet, H. T. Kung, and D. Cox, “Triton: An intermediate language and compiler for tiled neural network computations,” in *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages (MAPL)*, 2019, pp. 10–19.
- [3] T. Ge, T. Zhang, and H. Liu, “ngAP: Non-blocking large-scale automata processing on GPUs,” in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2024, pp. 1–15.
- [4] T. Mytkowicz, M. Musuvathi, and W. Schulte, “Data-parallel finite-state machines,” in *Proceedings of the 19th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2014, pp. 529–542.
- [5] V. Yaneva, A. Kapoor, A. Rajan, and C. Dubach, “Accelerated finite state machine test execution using gpus,” in *IEEE International Conference on Software Testing, Verification and Validation (ICST)*. IEEE, 2017, pp. 109–119.

- [6] X. Yao, Z. Li *et al.*, “HybridSA: GPU acceleration of multi-pattern regex matching using bit parallelism,” *Proceedings of the ACM on Programming Languages*, vol. 8, no. OOPSLA2, 2024.
- [7] T. Ge, T. Zhang, H. Liu *et al.*, “Interleaved bitstream execution for multi-pattern regex matching on GPUs,” in *Proceedings of the 58th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2025.